

Raport Techniczny: Ewaluacja Architektury Hybrydowej Web 2.5 dla Platformy TipJar+

1. Wstęp: Paradygmat Web 2.5 w Kontekście Ekonomii Twórców

Współczesna architektura aplikacji zdecentralizowanych (dApps) przechodzi fundamentalną transformację, ewoluując z purystycznego modelu Web 3.0, w którym każda warstwa stosu technologicznego musi być zdecentralizowana, w kierunku modelu hybrydowego, określanego mianem Web 2.5. Dla projektu takiego jak TipJar+, którego rdzeniem biznesowym jest obsługa mikropłatności i interakcji w czasie rzeczywistym pomiędzy twórcami a ich odbiorcami, wybór odpowiedniego fundamentu technologicznego nie jest jedynie decyzją inżynierską, lecz strategiczną determinującą rentowność, skalowalność oraz User Experience (UX).

Niniejszy raport stanowi wyczerpującą analizę komparatywną dwóch konkurencyjnych zestawów technologicznych. Pierwszy, reprezentujący podejście "Web 3.0 Native", opiera się na protokole The Graph do indeksowania danych, sieci Arweave do trwałego składowania treści oraz Cloudinary jako warstwie dostarczania. Drugi, proponowany jako alternatywa "Web 2.5", wykorzystuje Supabase jako suwerenny backend indeksujący, sieć Storj jako warstwę składowania kompatybilną z S3 oraz Cloudinary. Celem analizy jest zidentyfikowanie kompromisów między suwerennością danych a wydajnością operacyjną, ze szczególnym uwzględnieniem specyfiki aplikacji typu *tipping*, gdzie opóźnienia rzędu sekund mogą negatywnie wpłynąć na monetyzację twórcy.¹

Analiza opiera się na założeniu, że TipJar+ dąży do masowej adopcji, co wymaga od architektury zdolności do obsługi wysokiego wolumenu transakcji przy minimalnych kosztach krańcowych oraz zapewnienia responsywności interfejsu tożsamej z aplikacjami Web 2.0, przy jednoczesnym zachowaniu transparentności i niezmienności (immutability) warstwy rozliczeniowej opartej na blockchainie.

2. Warstwa Danych i Indeksowania: Konfrontacja Supabase i The Graph

Fundamentem każdej aplikacji blockchainowej jest mechanizm odczytu stanu łańcucha. Blockchainy, ze swej natury, są zoptymalizowane pod kątem zapisu i konsensusu, a nie wydajnego odczytu złożonych zapytań analitycznych. Dlatego też warstwa indeksująca staje

się krytycznym elementem infrastruktury.

2.1. The Graph: Ograniczenia Modelu Zdecentralizowanego w Aplikacjach Czasu Rzeczywistego

The Graph stał się standardem przemysłowym w ekosystemie Ethereum, oferując zdecentralizowany protokół indeksowania danych poprzez tzw. *Subgraphs*. Architektura ta polega na sieci niezależnych węzłów (Indexers), które są zachęcane ekonomicznie (poprzez token GRT) do indeksowania konkretnych podgrafów i obsługi zapytań.³ Choć model ten zapewnia wysoką odporność na cenzurę i brak pojedynczego punktu awarii, w kontekście aplikacji typu TipJar+ ujawnia istotne ograniczenia wydajnościowe.

Opóźnienia Synchronizacji i Determinizm

W modelu The Graph, indeksatorzy muszą czekać na finalizację bloku lub osiągnięcie odpowiedniego poziomu pewności (confirmation blocks), aby uniknąć indeksowania bloków, które mogą zostać odrzucone w wyniku reorganizacji łańcucha (reorg). Proces ten wprowadza inherentne opóźnienie (latency) między momentem wykonania transakcji on-chain a jej widocznością w API GraphQL. W sieciach o wysokiej przepustowości lub w momentach kongestii, opóźnienie to może wynosić od kilku do kilkudziesięciu sekund.⁵ Dla streamera używającego TipJar+, który oczekuje, że powiadomienie o donacji pojawi się na ekranie w momencie jej wystania przez widza, takie opóźnienie jest degradujące dla UX.

Model Kosztowy Oparty na Zapytaniach

Ekonomia The Graph opiera się na płatnościach za każde wykonane zapytanie. W przypadku aplikacji o wysokiej częstotliwości odświeżania danych (np. dashboard przychodów twórcy, pasek ostatnich donacji), koszty te stają się trudne do przewidzenia i skalują się liniowo wraz z ruchem użytkowników. Co więcej, konieczność zarządzania tokenem GRT i interakcji z rynkiem indeksatorów wprowadza dodatkową warstwę złożoności operacyjnej.⁵

2.2. Supabase: Suwerenny Indeks w Architekturze Web 2.5

Propozycja wykorzystania Supabase (PostgreSQL + narzędzia otaczające) jako warstwy indeksującej stanowi przesunięcie paradygmatu z "odczytu zaufanego" (trusted retrieval) na "odczyt wydajny" (performant retrieval). W tym modelu TipJar+ nie polega na zewnętrznej sieci indeksatorów, lecz buduje własny, suwerenny mechanizm ingestii danych.

Architektura Ingestii Sterowana Zdarzeniami

Wykorzystując **Supabase Edge Functions** (oparte na środowisku Deno), TipJar+ może wdrożyć architekturę sterowaną zdarzeniami (Event-Driven Architecture). Funkcje te mogą nasłuchiwać na Webhooki wysyłane przez dostawców węzłów RPC (takich jak Alchemy czy QuickNode) w momencie wystąpienia zdarzenia na obserwowanym smart kontrakcie.⁶ Alternatywnie, funkcje mogą cyklicznie odpytywać łańcuch (polling), co jednak jest mniej

efektywne.

Proces ten wygląda następująco:

1. Smart kontrakt emituje zdarzenie DonationReceived.
2. Dostawca RPC wykrywa zdarzenie i wysyła payload JSON do endpointu Supabase Edge Function.
3. Funkcja weryfikuje podpis kryptograficzny webhooka (aby zapobiec spoofingowi), parsuje dane i dokonuje atomowego zapisu do bazy PostgreSQL.⁷

Przewaga SQL i Supabase Realtime

Kluczową przewagą Supabase nad The Graph w kontekście TipJar+ jest wykorzystanie relacyjnej bazy danych PostgreSQL oraz modułu Realtime.

- **Analityka Finansowa:** SQL jest językiem znacznie potężniejszym analitycznie niż GraphQL. Pozwala na tworzenie skomplikowanych raportów przychodów, obliczanie średnich, czy segmentację darczyńców w czasie rzeczywistym, co w The Graph wymagałoby pobrania dużej ilości surowych danych i przetwarzania ich po stronie klienta.⁹
- **WebSockets:** Moduł Supabase Realtime, oparty na technologii Elixir/Phoenix, nasłuchuje zmian w dzienniku WAL (Write-Ahead Log) bazy danych i automatycznie wypycha aktualizacje do podłączonych klientów poprzez WebSockets.¹¹ Oznacza to, że w momencie zapisu transakcji do bazy przez Edge Function, interfejs streamera jest aktualizowany w czasie rzędu milisekund, co idealnie wpisuje się w wymagania aplikacji "live".

2.3. Tabela Porównawcza Warstwy Danych

Poniższa tabela syntetyzuje kluczowe różnice między analizowanymi podejściami, uwydatniając przewagi wydajnościowe modelu Web 2.5.

Cecha Architektoniczna	The Graph (Web 3.0 Native)	Supabase (Web 2.5 Hybrid)	Implikacja dla TipJar+
Model Dostępu do Danych	Pull/Query (GraphQL)	Push/Subscribe (SQL + Realtime)	Supabase eliminuje konieczność ciągłego odpytywania API (polling).
Opóźnienie	Sekundy/Minuty	Milisekundy	Krytyczne dla funkcji "live alerts"

(Latency)	(zależne od sieci)	(Realtime)	w TipJar+.
Obsługa Reorgów	Automatyczna (wbudowana w protokół)	Manualna (wymaga implementacji)	Supabase wymaga logiki weryfikującej stabilność bloku. ⁵
Koszt Operacyjny	Zmienny (opłata za zapytanie w GRT)	Przewidywalny (Compute + Storage)	Supabase oferuje lepszą kontrolę budżetową przy skalowaniu. ¹³
Elastyczność Zapytań	Ograniczona (schemat GraphQL)	Pełna (PostgreSQL, Joins, Views)	SQL umożliwia zaawansowaną analitykę biznesową (CRM twórcy).
Punkt Awarii	Zdecentralizowany (wiele węzłów)	Scentralizowany (Single Point of Failure)	Wymaga wdrożenia strategii High Availability i backupów w Supabase.

2.4. Wyzwanie Techniczne: Obsługa Reorganizacji Łańcucha (Chain Reorgs)

Jednym z najpoważniejszych wyzwań przy migracji na Supabase jest konieczność samodzielnego obsłużenia zjawiska reorganizacji łańcucha. W The Graph, jeśli blok zostanie wycofany, protokół automatycznie cofa zmiany w bazie danych subgrafu. W Supabase, jeśli Edge Function zapisze transakcję z bloku, który później stanie się "sierotą", dane w bazie będą niespójne ze stanem faktycznym łańcucha.⁵

Strategia Mitygacji dla TipJar+:

Rekomenduje się wdrożenie mechanizmu "Optimistic UI, Pessimistic Settlement".

1. **Stan Oczekujący:** Transakcja trafia do bazy ze statusem pending natychmiast po wykryciu (dla UX).
2. **Proces Potwierdzania:** Oddzielny proces (Background Worker w Edge Functions) sprawdza status transakcji po N potwierdzeniach (np. 12 bloków dla Ethereum) i zmienia status na confirmed.⁸
3. **Rollback:** W przypadku wykrycia reorgu (np. przez webhook reorg_detected z Alchemy), system musi być zdolny do usunięcia lub oznaczenia transakcji jako invalid i wysłania

korekty do klienta poprzez Realtime.

3. Warstwa Składowania: Ekonomia Trwałości – Arweave vs. Storj

Dla platformy TipJar+, która obsługuje media (wideo z podziękowaniami, grafiki NFT, treści ekskluzywne), wybór warstwy storage determinuje długoterminową rentowność. Porównujemy tu dwa radykalnie odmienne modele: Arweave ("zapłać raz, przechowuj na zawsze") oraz Storj ("płać za to, czego używasz").

3.1. Arweave: Kosztowna Wieczność i Permaweb

Arweave oferuje unikalną propozycję wartości w postaci *Permawebu*. Architektura ta opiera się na strukturze *Blockweave* i mechanizmie konsensusu SPoRA (Succinct Proofs of Random Access), który wymusza na górnikach przechowywanie historycznych danych w celu walidacji nowych bloków.¹⁴

Model Ekonomiczny Endowment

Cena przechowywania danych w Arweave zawiera w sobie składkę na "fundusz wieczysty" (endowment). Zakłada się, że koszt fizycznego nośnika danych (HDD/SSD) będzie spadał w czasie. Odsetki generowane przez fundusz mają pokrywać koszty przechowywania w nieskończoność. Choć idea ta jest rewolucyjna dla archiwizacji dziedzictwa kulturowego, w kontekście biznesowym TipJar+ generuje ogromny koszt początkowy (CAPEX).

Przechowywanie 1 TB danych na Arweave to koszt rzędu tysięcy dolarów (waha się w zależności od ceny tokena AR), płacone z góry.¹⁶

Ograniczenia Prawne i Operacyjne

Arweave jest z założenia niezmienny. Oznacza to, że raz wgranych treści nie da się usunąć. W kontekście regulacji takich jak RODO (GDPR) i "prawa do bycia zapomnianym", stanowi to istotne ryzyko prawne dla platformy operującej danymi użytkowników. Jeśli użytkownik zażąda usunięcia swoich danych, TipJar+ nie będzie w stanie tego wykonać na poziomie protokołu, co może prowadzić do komplikacji prawnych.¹⁸

3.2. Storj: Zdecentralizowana Chmura w Modelu Web 2.5

Storj reprezentuje podejście DCS (Decentralized Cloud Storage), które jest znacznie bliższe modelowi chmurowemu (AWS S3), zachowując jednak architekturę rozproszoną. Pliki są dzielone na fragmenty (Erasure Coding), szyfrowane i rozsyłane do tysięcy niezależnych węzłów na całym świecie. Do odtworzenia pliku potrzebna jest tylko część fragmentów (np. 29 z 80), co zapewnia wysoką dostępność i odporność na awarie węzłów.¹⁴

Kompatybilność z S3 jako Klucz do Integracji

Największą przewagą Storj w analizowanym stosie technologicznym jest natywna kompatybilność z protokołem S3. Storj udostępnia bramkę (Gateway MT), która pozwala aplikacjom "widzieć" sieć Storj jako zwykły bucket S3.

- **Łatwość Integracji:** Pozwala to na bezproblemowe podłączenie Storj do Cloudinary jako zewnętrznego źródła danych (o czym w kolejnym rozdziale) oraz używanie standardowych narzędzi deweloperskich (AWS CLI, SDK).¹⁹
- **Model Kosztowy OPEX:** Koszt przechowywania w Storj wynosi około **4 USD za TB miesięcznie**.¹⁶ Jest to model "Pay-as-you-go", który pozwala na skalowanie kosztów proporcjonalnie do wzrostu bazy użytkowników i przychodów, bez konieczności zamrażania kapitału na "wieczyste" przechowywanie danych, które mogą stracić na aktualności po miesiącu.

3.3. Tabela Porównawcza TCO (Total Cost of Ownership)

Poniższa tabela przedstawia symulację kosztów dla 10 TB danych w perspektywie 5 lat, uwzględniając specyfikę obu modeli.

Parametr Kosztowy	Arweave (Model Endowment)	Storj (Model Subskrypcyjny)	Wnioski dla TipJar+
Koszt Inicjalny (10 TB)	~\$21,300 - \$90,000+ (zależne od rynku)	\$40 (pierwszy miesiąc)	Storj drastycznie obniża bariery wejścia dla startupu.
Koszt 5-letni (Storage)	\$0 (opłacone z góry)	~\$2,400 (\$40 * 60 mc)	Storj pozostaje tańszy nawet w długim horyzoncie. ¹⁷
Koszt Transferu (Egress)	Zazwyczaj darmowy (przez bramki)	\$7/TB (pobieranie)	Koszt Egress w Storj mitygowany przez cache Cloudinary.
Usuwanie Danych	Niemożliwe (Immutability)	Możliwe (Standardowe DELETE)	Storj umożliwia zgodność z RODO i zarządzanie cyklem

			życia danych.
Prywatność	Publiczne (domyślnie)	Szyfrowane End-to-End	Storj lepszy dla treści premium/prywatnych w TipJar+.

4. Warstwa Dostarczania Mediów: Orkiestracja Cloudinary w Ekosystemie Hybrydowym

Cloudinary pełni w projekcie TipJar+ rolę silnika transformacji i optymalizacji mediów (DAM). Jego integracja z warstwą storage jest kluczowa dla wydajności (czas ładowania obrazów/wideo).

4.1. Wyzwania Integracji Cloudinary z Arweave (Fetch)

W architekturze opartej na Arweave, Cloudinary musi korzystać z mechanizmu "Remote Fetch", pobierając zasoby poprzez publiczne bramki HTTP (np. arweave.net/TX_ID).

- **Wąskie Gardło Bramek:** Publiczne bramki Arweave często nakładają limity zapytań (rate limiting) lub ulegają przeciążeniom. Jeśli Cloudinary nie zdoła pobrać pliku źródłowego w określonym czasie (timeout), użytkownik końcowy otrzyma błąd 404/500, co jest niedopuszczalne w produkcji.²¹
- **Brak Kontroli Dostępu:** Ponieważ URL do zasobu w Arweave jest publiczny, każdy kto zna ID transakcji, może pobrać plik z pominięciem aplikacji TipJar+, co uniemożliwia skuteczną monetyzację treści ekskluzywnych (Token-Gated Content) bez dodatkowej warstwy szyfrowania na poziomie pliku.

4.2. Synergia Cloudinary i Storj poprzez Protokół S3

Dzięki kompatybilności Storj z S3, integracja z Cloudinary wchodzi na wyższy poziom bezpieczeństwa i wydajności. Cloudinary pozwala na zdefiniowanie "Własnego Źródła S3" (Custom S3 Source), mapując bucket Storj jako prywatny zasób.²²

Architektura Przepływu Danych (Media Pipeline):

1. **Upload:** Użytkownik (twórca) przesyła plik. Aplikacja frontendowa prosi Supabase Edge Function o wygenerowanie **Presigned URL** do bucketu Storj.
2. **Bezpośredni Transfer:** Przeglądarka wysyła plik bezpośrednio do węzłów Storj (omijając serwery TipJar+, co oszczędza pasmo).²³
3. **Indeksacja:** Po udanym uploadzie, Edge Function zapisuje metadane pliku (klucz S3) w bazie Supabase.

4. **Dostarczanie:** Gdy widz chce zobaczyć treść, frontend generuje URL Cloudinary wskazujący na ten plik. Cloudinary, używając swoich poświadczeń (Access/Secret Key), pobiera plik z prywatnego bucketu Storj, optymalizuje go (np. transkoduje wideo do HLS, zmienia format obrazu na AVIF) i serwuje przez swój CDN.²⁵

Zalety tego podejścia:

- **Security:** Pliki źródłowe w Storj są niedostępne publicznie. Dostęp ma tylko Cloudinary.
- **Wydajność:** Storj, dzięki pobieraniu równoległemu (parallelism) segmentów pliku z wielu węzłów, potrafi nasycić łącze szybciej niż pojedynczy serwer HTTP, co przyspiesza proces "Cold Start" (pierwszego pobrania) przez Cloudinary.²⁰

5. Bezpieczeństwo i Suwerenność w Modelu Web 2.5

Przejście na Supabase i Storj przesuwą środek ciężkości w stronę centralizacji (Web 2.5). Kluczowe jest zrozumienie i mitygacja ryzyk z tym związanych.

5.1. Hybrydowa Autentykacja: Most między Portfelem a Bazą Danych

W czystym Web3 portfel jest jedynym identyfikatorem. W Web 2.5 musimy powiązać ten portfel z sesją w tradycyjnej bazie danych. Supabase Auth wspiera natywnie standard **Sign In With Ethereum (SIWE)**.²⁷

Implementacja SIWE i Row Level Security (RLS)

Proces logowania polega na podpisaniu przez użytkownika wiadomości kryptograficznej (challenge). Supabase weryfikuje ten podpis i wydaje token JWT (JSON Web Token).

- **RLS jako Strażnik:** Token JWT zawiera adres portfela użytkownika. Dzięki mechanizmowi Row Level Security w PostgreSQL, możemy definiować polityki dostępu bezpośrednio w bazie danych. Przykładowo: `CREATE POLICY "User can update own profile" ON profiles FOR UPDATE USING (auth.uid() = wallet_address)`.
- **Implikacja:** Nawet jeśli API Supabase zostanie wystawione publicznie, baza danych sama w sobie egzekwuje uprawnienia oparte na kryptografii blockchainowej. Jest to potężne połączenie elastyczności SQL z bezpieczeństwem Web3.²⁹

5.2. Ryzyko Centralizacji i Strategia Wyjścia (Exit Strategy)

Użycie Supabase (jako usługi hostowanej) i Storj (zarządzanego przez Satelity Storj Labs) wprowadza ryzyko cenzury lub awarii platformy.

- **Supabase:** Jest to narzędzie open-source. TipJar+ może wdrożyć strategię **"Sovereign Backup"**, polegającą na regularnym rzuceniu bazy danych (`pg_dump`) i gotowości do uruchomienia własnej instancji Supabase (Docker) na niezależnej infrastrukturze w ciągu kilku godzin. Kod Edge Functions jest przenośny (Deno/Node.js).³⁰

- **Storj:** Choć Satelity są punktem centralnym metadanych, sama sieć jest zdecentralizowana. Ryzyko cenzury jest niższe niż w AWS, ale wyższe niż w Arweave. Dla TipJar+ jest to akceptowalne ryzyko w zamian za zgodność z regulacjami i możliwość moderacji treści nielegalnych.¹⁸

6. Podsumowanie i Rekomendacja Strategiczna

Analiza zestawów technologicznych w kontekście wymagań projektu TipJar+ prowadzi do jednoznacznej konkluzji na korzyść architektury hybrydowej Web 2.5.

Rekomendacja: Wdrożenie stosu **Supabase + Cloudinary + Storj**.

Uzasadnienie Strategiczne:

1. **Dominacja UX:** Użytkownicy oczekują natychmiastowej reakcji interfejsu (Realtime), którą gwarantuje Supabase, a której nie jest w stanie zapewnić The Graph bez istotnych opóźnień. W ekonomii twórców, responsywność przekłada się bezpośrednio na skłonność do donacji.
2. **Efektywność Kapitałowa:** Model płatności Storj (OPEX) drastycznie obniża bariery wejścia i ryzyko finansowe w porównaniu do modelu Arweave (CAPEX), co jest kluczowe dla startupu w fazie wzrostu.
3. **Pragmatyzm Technologiczny:** Wykorzystanie standardów przemysłowych (SQL, S3) zamiast niszowych (GraphQL, Arweave Transaction Tags) ułatwia rekrutację talentów, integrację narzędzi i rozwój produktu.

Kompromis: Decyzja ta oznacza rezygnację z absolutnej decentralizacji i "nieśmiertelności" danych na rzecz wydajności i kontroli. Jest to jednak kompromis świadomy, mitygowany przez otwartość kodu Supabase, szyfrowanie Storj i kryptograficzną autentykację SIWE. W obecnej fazie rozwoju Internetu, Web 2.5 stanowi optymalną ścieżkę dla aplikacji, które chcą dostarczyć wartość masowemu użytkownikowi, zachowując ducha własności Web3.

Cytowane prace

1. What is Web 2.5? Bridging the Gap Between Web 2.0 and Web 3.0 - STL Digital, otwierano: grudnia 7, 2025, <https://www.stldigital.tech/blog/what-is-web-2-5-bridging-the-gap-between-web-2-0-and-web-3-0/>
2. Why We Should Invest in Web 2.5: The Real-World Applications of Blockchain Technology, otwierano: grudnia 7, 2025, <https://m-tiesler.medium.com/why-we-should-invest-in-web-2-5-the-real-world-applications-of-blockchain-technology-a59f29ca005a>
3. Across vs Coinbase vs Subgraphs Comparison | Builder's Guide - Quicknode, otwierano: grudnia 7, 2025, <https://www.quicknode.com/builders-guide/compare/across-by-risk-labs-vs-coinbase>

- [base-by-coinbase-inc-vs-subgraphs-by-goldsky](#)
4. Top Web3 Data Indexers: Empowering Your Web3 Development | by Reactive Network, otwierano: grudnia 7, 2025,
<https://medium.com/parsiq/top-web3-data-indexers-empowering-your-web3-development-b6ce609ea92c>
 5. A better indexer compared to thegraph? : r/ethdev - Reddit, otwierano: grudnia 7, 2025,
https://www.reddit.com/r/ethdev/comments/1404f3b/a_better_indexer_compared_to_thegraph/
 6. Edge Functions | Supabase Docs, otwierano: grudnia 7, 2025,
<https://supabase.com/docs/guides/functions>
 7. Edge Functions Architecture | Supabase Docs, otwierano: grudnia 7, 2025,
<https://supabase.com/docs/guides/functions/architecture>
 8. Handling Stripe Webhooks | Supabase Docs, otwierano: grudnia 7, 2025,
<https://supabase.com/docs/guides/functions/examples/stripe-webhooks>
 9. Supabase and graph databases? - Reddit, otwierano: grudnia 7, 2025,
https://www.reddit.com/r/Supabase/comments/1hserm7/supabase_and_graph_databases/
 10. How I Created Superior RAG Retrieval With 3 Files in Supabase - Reddit, otwierano: grudnia 7, 2025,
https://www.reddit.com/r/Supabase/comments/1ovj8rh/how_i_created_superior_rag_retrieval_with_3_files/
 11. Realtime: Multiplayer Edition - Supabase, otwierano: grudnia 7, 2025,
<https://supabase.com/blog/supabase-realtime-multiplayer-general-availability>
 12. Realtime Architecture | Supabase Docs, otwierano: grudnia 7, 2025,
<https://supabase.com/docs/guides/realtime/architecture>
 13. The Complete Guide to Supabase Pricing Models and Cost Optimization - Flexprice, otwierano: grudnia 7, 2025,
<https://flexprice.io/blog/supabase-pricing-breakdown>
 14. Who Has Better Prospects in Decentralized Storage: Arweave (AR) or Storj? | 傻爷说币 on Binance Square, otwierano: grudnia 7, 2025,
<https://www.binance.com/en-IN/square/post/27753637705553>
 15. Who Has Better Prospects in Decentralized Storage: Arweave (AR) or Storj? | 傻爷说币 on Binance Square, otwierano: grudnia 7, 2025,
<https://www.binance.com/en/square/post/27753637705553>
 16. Centralized vs Decentralized Storage Cost (2023) - CoinGecko, otwierano: grudnia 7, 2025,
<https://www.coingecko.com/research/publications/centralized-decentralized-storage-cost>
 17. Filecoin vs Sia vs Storj economic values for node runners and the longevity of the network : r/CryptoTechnology - Reddit, otwierano: grudnia 7, 2025,
https://www.reddit.com/r/CryptoTechnology/comments/1asx21x/filecoin_vs_sia_vs_storj_economic_values_for_node/
 18. A Comparison of the Top 6 Decentralized Storage Networks | by SidCord | Medium, otwierano: grudnia 7, 2025,

<https://medium.com/@sidcord/a-comparison-of-the-top-6-decentralized-storage-networks-7efbd3f5467a>

19. S3 Compatibility - Storj Docs, otwierano: grudnia 7, 2025, <https://storj.dev/dcs/api/s3/s3-compatibility>
20. What is S3 Compatibility? - Storj, otwierano: grudnia 7, 2025, <https://www.storj.io/blog/what-is-s3-compatibility>
21. NFT API FAQ | Alchemy Docs, otwierano: grudnia 7, 2025, <https://www.alchemy.com/docs/reference/nft-api-faq>
22. How do I allow Cloudinary to read assets from my private S3 bucket?, otwierano: grudnia 7, 2025, <https://support.cloudinary.com/hc/en-us/articles/203276521-How-do-I-allow-Cloudinary-to-read-assets-from-my-private-S3-bucket>
23. Is it bad/against terms/immoral to use (free) cloudinary for upload/image manipulation but then to fetch the images for storage/distribution on your own S3/CloudFront setup? - Quora, otwierano: grudnia 7, 2025, <https://www.quora.com/Is-it-bad-against-terms-immoral-to-use-free-cloudinary-for-upload-image-manipulation-but-then-to-fetch-the-images-for-storage-distribution-on-your-own-S3-CloudFront-setup>
24. How to upload an image file from an s3 bucket to cloudinary (nodejs) - Stack Overflow, otwierano: grudnia 7, 2025, <https://stackoverflow.com/questions/74862988/how-to-upload-an-image-file-from-an-s3-bucket-to-cloudinary-nodejs>
25. S3 compatible storage | Storj Object Storage, otwierano: grudnia 7, 2025, <https://www.storj.io/object-storage/s3-compatible-storage>
26. Set up Cloudinary to read from a private S3 Bucket - Charlie Spalevic, otwierano: grudnia 7, 2025, <https://www.cspalevic.com/blog/cloudinary-s3-connection>
27. Sign in with Web3 | Supabase Docs, otwierano: grudnia 7, 2025, <https://supabase.com/docs/guides/auth/auth-web3>
28. gm web3, welcome aboard to Sign in with Web3 (Solana, Ethereum) - Supabase, otwierano: grudnia 7, 2025, <https://supabase.com/blog/login-with-solana-ethereum>
29. Subverting Web2 Authentication in Web3, otwierano: grudnia 7, 2025, <https://osec.io/blog/2025-03-07-subverting-web2-authentication-in-web3/>
30. Supabase vs. MongoDB: a Complete Comparison in 2025 - Bytebase, otwierano: grudnia 7, 2025, <https://www.bytebase.com/blog/supabase-vs-mongodb/>
31. Supabase Edge Functions - Deploy JavaScript globally in seconds, otwierano: grudnia 7, 2025, <https://supabase.com/edge-functions>
32. End Users Are Ready for Web3—What App Developers Need to Know - Storj, otwierano: grudnia 7, 2025, <https://www.storj.io/blog/end-users-are-ready-for-web3-what-app-developers-need-to-know>